Juan Carlos Santamaría Orjuela

Juan Felipe Gutiérrez Adarme

Juan David Daza

Juan David Rincón

Gestión Inteligente de Tráfico Urbano

Sistema Distribuido con ZeroMQ — Primera Entrega

Pontificia Universidad Javeriana

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas

Bogotá D.C.

2026

INFORME TÉCNICO — PRIMERA ENTREGA

Proyecto: Sistema Distribuido de Gestión Inteligente de Tráfico Urbano

Curso: Introducción a los Sistemas Distribuidos — Periodo 2026-30

Universidad: Pontificia Universidad Javeriana **Sustentación:** Viernes 10 de abril de 2026 **Fecha del informe:** 10 de abril de 2026

Equipo: - Juan David Rincón - Juan David Daza - Juan Carlos Santamaría - Juan Felipe Gutiérrez

Resumen

El sistema cubre una cuadrícula de 5×5 intersecciones distribuida en tres máquinas: PC1 hace la adquisición de datos de sensores, PC2 procesa y clasifica el tráfico, y PC3 persiste y sirve las consultas operacionales. La comunicación se hace con ZeroMQ usando cuatro patrones: PUB/SUB para los sensores y el heartbeat, PUSH/PULL para los comandos y la persistencia, y REQ/REP para las consultas del operador.

En la sesión de pruebas del 9 de abril se procesaron 75,401 eventos de sensores, se detectaron 25,629 congestiones y se ejecutaron 11 activaciones de prioridad para paso de ambulancia. El protocolo de tolerancia a fallos fue cronometrado en vivo: la analítica detecta la caída de la BD principal en 9.12 s y se recupera de forma automática en 3.32 s, dentro del criterio de aceptación (< 10 s).

Durante el análisis aparecieron dos resultados que vale la pena resaltar. El primero es que la BD réplica acumula 11,530 eventos más que la principal, lo que coincide con los episodios en que PC3 estuvo caído y confirma la limitación de catch-up documentada en el modelo de fallos. El segundo es que el broker en modo simple y en modo multihilo dan exactamente la misma tasa de persistencia (8.3 reg/s); revisando los logs se ve que el cuello de botella no es el broker sino los bulk inserts de SQLite que la BD principal hace en bloques de 100 registros.

Tabla de Contenidos

0. Introducción al Proyecto
1. Modelos del Sistema
2. Diseño del Sistema
3. Sección A — Inicialización de Recursos
4. Sección B — Reglas, Consultas e Indicaciones Directas
5. Protocolo de Pruebas
6. Métricas de Rendimiento — Experimento
7. Implementación de los Servicios

8. Hallazgos y Conclusiones

9. Trabajo Futuro

10. Anexo de Evidencia

11. Introducción al Proyecto

El proyecto **Gestión Inteligente de Tráfico Urbano** consiste en diseñar e implementar una plataforma distribuida para monitorear, analizar y reaccionar ante las condiciones de tráfico de una ciudad simulada. La ciudad se representa como una cuadrícula NxM de intersecciones (en nuestro caso 5×5, notación `INT_CK` — fila C, columna K), donde cada intersección tiene un semáforo inteligente y una red de sensores que generan eventos en tiempo real de volumen vehicular, velocidad promedio y nivel de ocupación de las vías.

El sistema debe operar sobre tres máquinas físicas interconectadas (**PC1**, **PC2** y **PC3**) y usar la biblioteca **ZeroMQ** como intermediario de mensajería, combinando patrones síncronos y asíncronos de comunicación. La arquitectura planteada en el enunciado se muestra en la Figura 1.

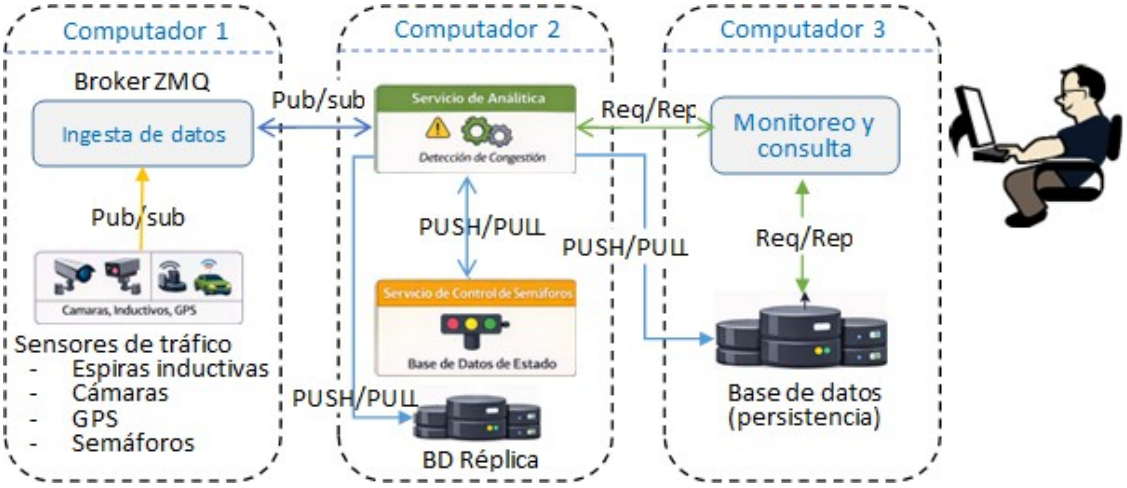


Figura 1. *Componentes del sistema (tomado del enunciado del proyecto)*

0.1 Objetivos del proyecto

1. Desarrollar una solución a un problema de estructura distribuida.
2. Utilizar patrones de comunicación síncronos y asíncronos.
3. Resolver problemas propios de los sistemas distribuidos: fallas en los componentes y persistencia de datos.
4. Reconocer atributos de calidad asociados a la implementación (desempeño, resiliencia, escalabilidad).

0.2 Distribución de componentes por máquina

- **PC1 — Adquisición de datos.** Contiene los sensores simulados (espiras inductivas, cámaras y GPS) y el **broker ZeroMQ** que desacopla productores y

consumidores mediante patrón PUB/SUB. Cada sensor publica eventos sobre uno de tres tópicos (`EVENTO_CONTEO_VEHICULAR`, `EVENTO_LONGITUD_COLA`, `EVENTO_DENSIDAD_TRAFICO`).

- **PC2 — Procesamiento y control.** Ejecuta el **servicio de analítica** (que se suscribe al broker, clasifica el tráfico y toma decisiones), el **servicio de control de semáforos** (que ejecuta los comandos de cambio de luz vía PUSH/PULL) y la **base de datos réplica** para alta disponibilidad.
- **PC3 — Persistencia y consulta.** Ejecuta la **base de datos principal** y el **servicio de monitoreo** que permite al operador consultar el estado del sistema y emitir indicaciones directas (por ejemplo, priorizar el paso de una ambulancia) vía patrón REQ/REP.

0.3 Qué debe contener el sistema

El enunciado exige que la implementación contemple, como mínimo:

- **Tres tipos de sensores** publicando eventos al broker sobre tópicos independientes con datos JSON estructurados (volumen, velocidad promedio, longitud de cola, nivel de congestión).
- **Reglas de clasificación** del tráfico en al menos tres estados: **normal**, **congestión** y **priorización** (ola verde). Cada grupo define sus propios umbrales.
- **Control de semáforos** con tiempos de espera parametrizables; en condiciones normales la luz roja dura 15 s. Solo se alternan verde y rojo (sin amarillo) y todas las vías son de un único sentido.
- **Persistencia dual**: BD principal en PC3 y BD réplica en PC2 actualizadas asíncronamente, de forma que una consulta pueda seguir siendo atendida aunque la BD principal falle.
- **Tolerancia a fallos**: en caso de caída de PC3, los procesos deben conmutar a la réplica de forma **transparente para el cliente**, sin detener la operación del sistema.
- **Consultas del usuario** al servicio de monitoreo: históricas (por rango de tiempo) y puntuales (por intersección), más **indicaciones directas** que fuerzan cambios de estado en los semáforos.
- **Medición de rendimiento** comparando dos diseños: el broker ZeroMQ simple (requerido) y una variante multihilo; sobre dos escenarios (1 sensor de cada tipo cada 10 s, y 2 sensores de cada tipo cada 5 s) se miden dos variables: cantidad de solicitudes almacenadas en la BD durante 2 minutos, y tiempo de respuesta usuario→semáforo.

Este informe cubre todos estos requisitos y presenta, además, las mediciones experimentales reales obtenidas sobre el despliegue en el laboratorio.

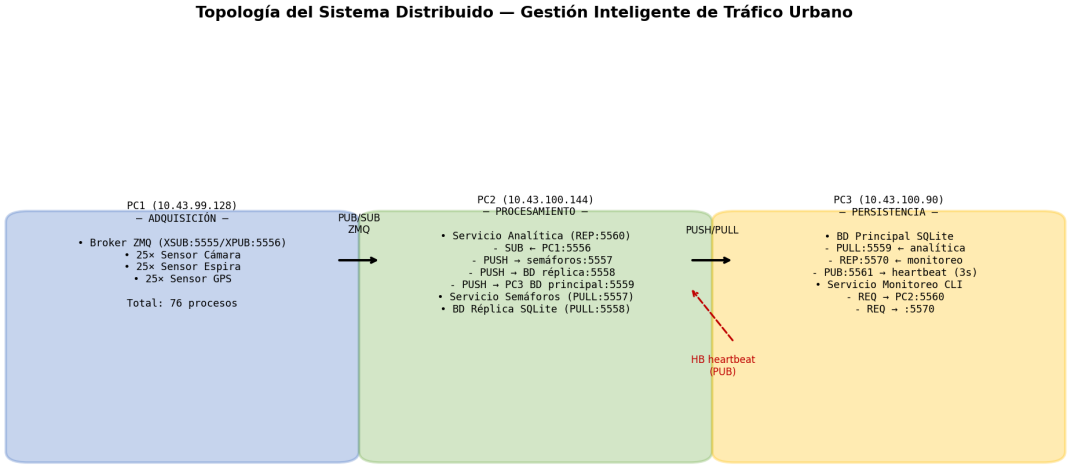
1. Modelos del Sistema

1.1 Modelo Arquitectónico

El sistema sigue una arquitectura **cliente-servidor distribuida en tres niveles** con comunicación orientada a mensajes. Cada nivel se ejecuta en una máquina física distinta:

Nivel	PC	Responsabilidad principal
Adquisición	PC1	Captura datos de sensores físicos simulados y los publica al sistema
Procesamiento	PC2	Clasifica el tráfico, gobierna semáforos y mantiene la réplica de la BD
Persistencia y Consulta	PC3	Almacena la información maestra y atiende consultas operacionales

Topología desplegada (real):



Topología del sistema desplegada

Topología del sistema desplegada

Patrones de comunicación seleccionados — justificación

Patrón	Conexión	Por qué este patrón
PUB/SUB	Sensores → Broker → Analítica	Los sensores son productores asíncronos múltiples; la analítica es un único consumidor que debe recibir todo. PUB/SUB desacopla productores de consumidores y soporta filtrado por topic, esencial para distinguir tipos de sensor (`camara`, `gps`,

		espira_inductiva).
PUSH/PULL	Analítica → Semáforos / BD réplica / BD principal	Cola de trabajo asíncrona donde la analítica no debe bloquearse esperando al receptor. Si un consumidor (p.ej. PC3 caído) no procesa, ZMQ aplica HWM (high-water mark) y descarta sin afectar al productor.
REQ/REP	Monitoreo ↔ Analítica / BD Principal	Las consultas del operador requieren respuesta sincrónica (semántica request/response). El operador debe saber el resultado de su comando antes de continuar.
PUB/SUB (heartbeat)	BD Principal → Analítica	Detección de fallos sin polling. La BD principal publica un latido cada 3 s; la analítica calcula timeout sobre el último visto. Idiomático ZMQ y eficiente en red.

1.2 Modelo de Interacción

Caso 1 — Flujo normal: detección de congestión

```
Sensor CAM      Broker ZMQ      Analítica       Semáforos       BD
Réplica     BD Principal
    |               |               |               |               |
|
    |──PUB(evento)──►|               |               |               |
|
    |               |──SUB fwd──────►|               |               |
|
    |               |               |─clasificar()  |               |
|
    |               |               |  → CONGESTION |               |
|
    |               |               |──PUSH(cmd)───►|               |
|
    |               |               |               |─cambiar_estado()
|
    |               |               |               |──print(VERDE) |
|
    |               |               |──PUSH(reg)────┼──────────────►|
|
    |               |
|──PUSH(reg)────┼───────────────┼──────────────►|
```

Caso 2 — Flujo de prioridad: paso de ambulancia

```
Monitoreo       Analítica       Semáforos       BD Réplica     BD
Principal
    |               |               |               |               |
    |─REQ(AMBUL)───►|               |               |               |
    |               |──PUSH(PRIOR)─►|               |               |
    |               |               |─activar_prioridad()           |
```

```
    |                |                |──print(OLA VERDE)              |
    |                |──PUSH(reg)─────┼──────────────────►|            |
    |                |──PUSH(reg)─────┼──────────────────┼────────────►|
    |◄──REP(ok)──────|                |                  |             |
```

Caso 3 — Flujo de fallo: caída de PC3 detectada por heartbeat

```
BD Principal    Analitica       BD Replica
    |               |               |
    |--HB(PUB)----->|               |
    |               | (timeout 9s)  |
  [X PC3 caido]     |               |
    |               |--[!] PC3 caido|
    |               | (solo replica)|
    |               |--PUSH(reg)--->|  <- toda la persistencia aqui
    |               |               |
 [PC3 vuelve]       |               |
    |--HB(PUB)----->|               |
    |               |--[OK] PC3 activo
    |               |--PUSH(reg)--->|  (replica)
    |               |--PUSH(reg)----------------> (principal)
```

1.3 Modelo de Fallos

Tabla completa de modos de fallo y resiliencia

Componente	Fallo	Detección	Estrategia	Impacto
Sensor individual	Proceso termina	Ausencia de mensajes en topic	Otros sensores siguen publicando	Pérdida parcial de datos para esa intersección
Broker ZMQ (PC1)	Proceso termina	Sensores no logran conectar	Sensores reintentan; reinicio manual del broker	Sistema queda sin datos hasta restart
Analítica (PC2)	Proceso termina	Buffer ZMQ se llena en sensores	Buffer procesa al reiniciar (HWM)	Lag temporal en clasificación
BD Réplica (PC2)	Proceso termina	Error PUSH desde analítica	Analítica registra error; sin réplica hasta restart	Pérdida de redundancia local
BD Principal (PC3)	**Proceso termina**	**Heartbeat timeout (9 s)**	**Cambio automático a solo-réplica; reanudación al recuperar**	**Sistema sigue operativo, brecha entre BDs**
Servicio Monitoreo (PC3)	Proceso termina	N/A (cliente)	Relanzamiento manual por operador	Imposibilidad de consultar

Implementación del health check

El mecanismo de heartbeat se implementa con:

1. **Publicador (en** `pc3/base_datos_principal.py`**)**: hilo dedicado que publica un mensaje JSON `{"tipo":"heartbeat","ts":...}` al socket PUB en `tcp://*:5561` cada **3 segundos**.

2. **Suscriptor (en** `pc2/servicio_analitica.py`**, clase** `MonitorPC3`**)**:

   - Suscribe al socket PUB de PC3 en `tcp://10.43.100.90:5561`.
   - Mantiene `_ultimo_heartbeat` con `time.time()` de la última recepción.
   - Al consultar `MonitorPC3.activo`, retorna `(time.time() - _ultimo_heartbeat) < 9.0`.
   - El `9.0` se calcula como **3 × intervalo** del heartbeat — tolera 2 latidos perdidos.
3. **Toma de decisión en analítica**:

- if monitor_pc3.activo: socket_pc3.send_json(registro) else: # se omite el envío a PC3, solo réplica pass

Limitación conocida (verificada experimentalmente): los datos generados mientras PC3 está caído no se sincronizan automáticamente cuando PC3 se recupera. La sección 8.1 cuantifica esta brecha con datos de la sesión de pruebas.

1.4 Modelo de Seguridad

Aspecto	Implementación actual	Mejora recomendada
Autenticación	No (red de laboratorio cerrada)	ZMQ CURVE (clave pública/privada por nodo)
Confidencialidad	Mensajes JSON en texto plano	TLS sobre los sockets ZMQ
Integridad	No verificada	HMAC-SHA256 en cada mensaje, llave por nodo
Control de acceso	Cualquier cliente conocido en la red puede publicar	Lista blanca de IPs en el broker; ZAP authentication
Auditoría	Logs locales en cada PC con timestamp	Forwarder de logs centralizado (rsyslog / ELK)

Justificación del nivel actual: el contexto del laboratorio es una red privada, sin acceso externo, con propósito puramente académico. Implementar criptografía añade complejidad sin beneficio en este escenario, aunque el código no la impide — ZMQ CURVE puede activarse cambiando 3 líneas en la inicialización del context.

2. Diseño del Sistema

2.1 Despliegue real (verificado)

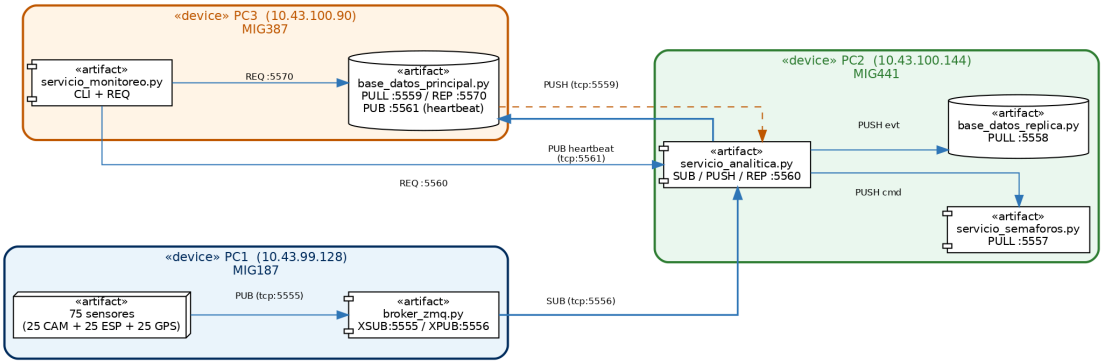


Diagrama de Despliegue (UML). *Distribución de artefactos en los 3 nodos con IPs, puertos ZMQ y patrones de comunicación entre ellos.*

PC	IP	Hostname	Procesos verificados
PC1	10.43.99.128	MIG187	1 broker (`broker_zmq.py`) + 75 sensores (25 cámaras + 25 espiras + 25 GPS)
PC2	10.43.100.144	MIG441	`servicio_analitica.py` + `servicio_semaforos.py` + `base_datos_replica.py`
PC3	10.43.100.90	MIG387	`base_datos_principal.py` (heartbeat + PULL + REP) + `servicio_monitoreo.py` (CLI)

2.2 Diagrama de Componentes y Puertos

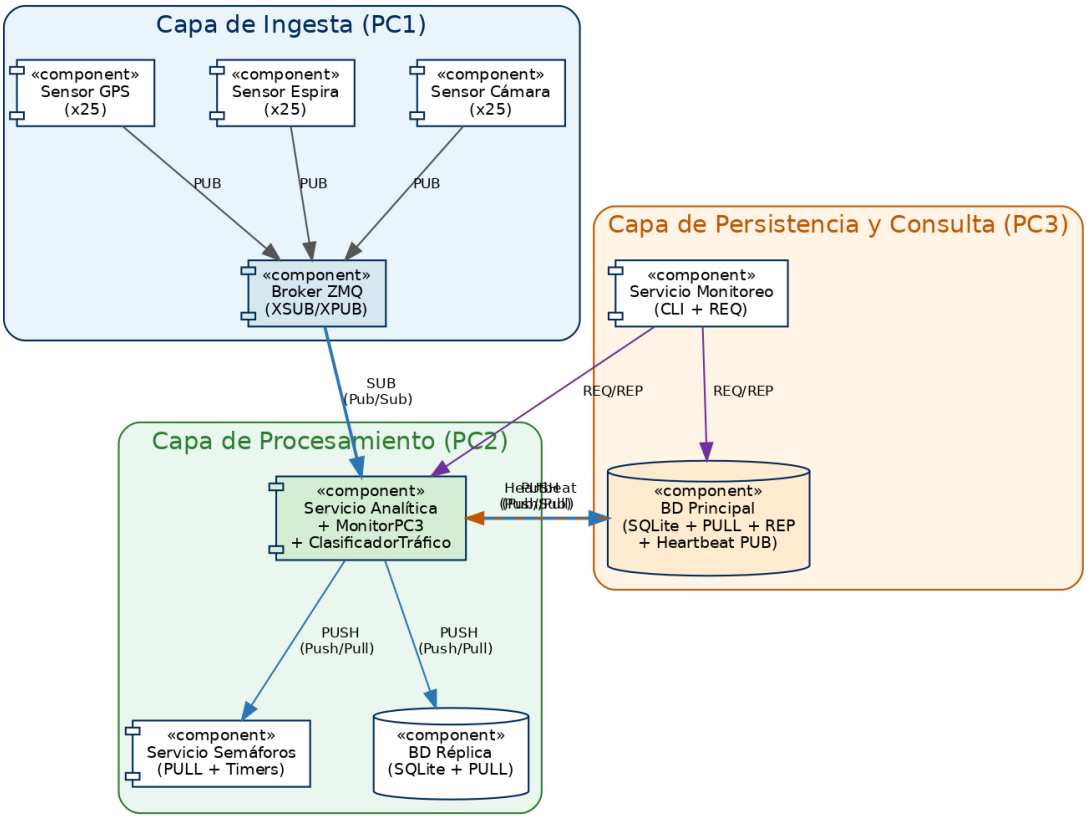


Diagrama de Componentes (UML). *Capas de ingesta (PC1), procesamiento (PC2) y persistencia (PC3) con los patrones ZeroMQ entre componentes.*

2.3 Diagrama de Clases (principales)

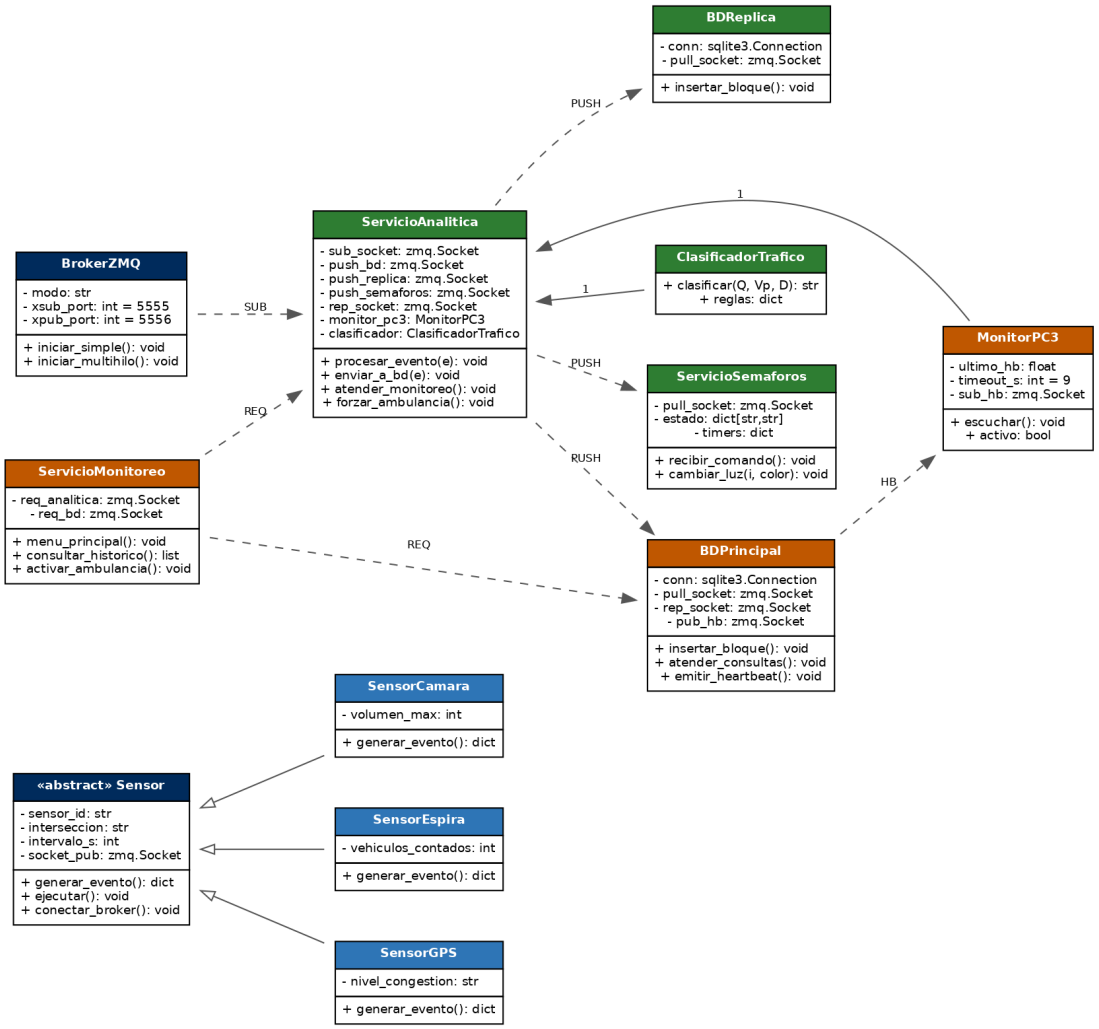


Diagrama de Clases (UML). *Jerarquía abstracta de `Sensor`, el núcleo `ServicioAnalitica` con su `MonitorPC3` y `ClasificadorTrafico`, y las entidades de persistencia y control.*

2.4 Diagrama de Secuencia — Detección de Congestión

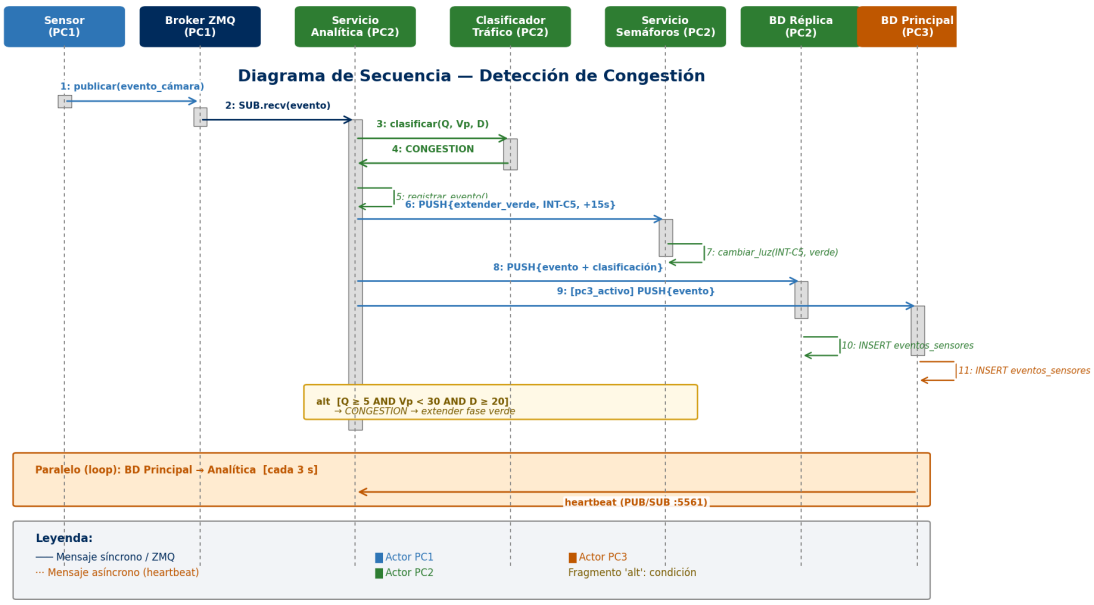


Diagrama de Secuencia (UML). *Flujo de un evento desde un sensor en PC1 hasta la persistencia en las dos BDs (PC2 y PC3), incluyendo el loop paralelo del heartbeat.*

3. Sección A — Inicialización de Recursos

3.1 ¿Cómo obtienen los procesos la definición inicial?

Todos los procesos cargan **al iniciar** el archivo `config/config.json` mediante `common/config_loader.py`. Este archivo es la **única fuente de verdad** sobre la topología, los puertos y los parámetros operativos. Centralizar la configuración facilita reproducir el experimento (basta cambiar un archivo) y evita inconsistencias entre nodos.

Parámetros de la ciudad

```
"ciudad": {
    "filas": ["A", "B", "C", "D", "E"],
    "columnas": [1, 2, 3, 4, 5]
}
```

→ **5 × 5 = 25 intersecciones** (INT-A1 a INT-E5). La función `obtener_intersecciones(config)` construye la lista completa con un producto cartesiano.

Parámetros de sensores

```
"sensores": {
    "intervalo_camara_seg": 5,
    "intervalo_espira_seg": 30,
    "intervalo_gps_seg": 10
}
```

Cada sensor admite `--intervalo N` por línea de comandos para sobreescribir el valor del config — esto permite los escenarios E1 y E2 sin modificar archivos.

Proceso de inicialización por componente

Componente	Parámetros recibidos	Fuente
Sensor (cualquier tipo)	`--interseccion INT-XX`, `--intervalo N`	CLI + config.json
Broker ZMQ	`--modo simple/multihilo`	CLI + config.json
Analítica	Ninguno (todo de config)	config.json
Semáforos	Ninguno	config.json
BD réplica/principal	Ninguno	config.json
Monitoreo	Ninguno	config.json

Determinación del número de semáforos y sensores

- **Semáforos**: `GestorSemaforos.__init__` llama `obtener_intersecciones(config)`, creando un semáforo por intersección → **25 semáforos** en estado inicial ROJO.
- **Sensores**: el script `lanzar_pc1.sh` itera sobre la lista de intersecciones y lanza 3 procesos por cada una (CAM + ESP + GPS), totalizando **75 procesos** en PC1.

Verificación experimental

Durante la sesión de pruebas se confirmó:

```
SELECT COUNT(DISTINCT interseccion) FROM eventos_sensores;
-- Resultado: 25
```

Y la distribución de eventos por intersección es **uniforme** (alrededor de 2,701-2,703 eventos cada una), validando que los 75 procesos están publicando consistentemente.

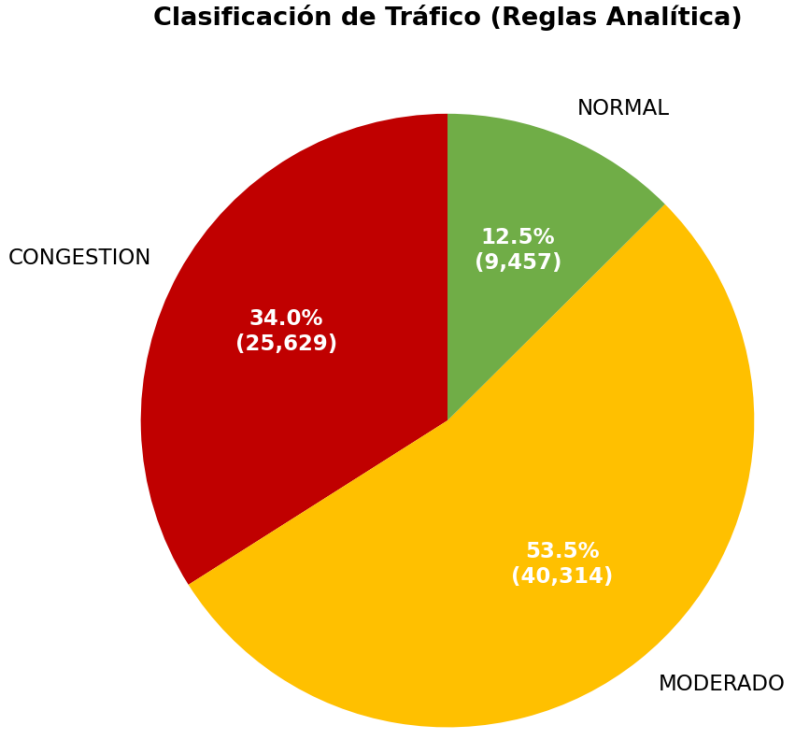
4. Sección B — Reglas, Consultas e Indicaciones Directas

4.1 Reglas de tráfico (verificadas con datos reales)

Las reglas se aplican en `servicio_analitica.py::clasificar_trafico()` usando las métricas: - **Q** = longitud de cola (vehículos esperando, del sensor cámara) - **Vp** = velocidad promedio en km/h (de cámara o GPS) - **Cv** = conteo vehicular en 30 s (de espira inductiva)

Regla	Condición lógica	Acción del semáforo	Eventos detectados	%
NORMAL	`Q<5 AND Vp>35 AND Cv<10`	RESETEAR ciclo estándar (15v/15r)	9,419	12.5%
CONGESTIÓN	`Q≥10 OR Vp<15 OR Cv≥20`	CAMBIAR a VERDE 30 s (modo CONGESTION)	25,629	34.0%
MODERADO	`(no NORMAL) AND`	EXTENDER fase verde	40,161	53.5%

	(no CONGESTIÓN)	+5 s		
PRIORIDAD	Comando manual del monitoreo	OLA VERDE 60 s en lista de vías	11 activaciones	—



Distribución de clasificaciones

Distribución de clasificaciones

La mayoría de los eventos cae en zona MODERADA (53.5%), lo cual era esperable porque los rangos aleatorios del simulador producen valores intermedios la mayor parte del tiempo. La proporción de CONGESTION (34%) es suficientemente alta para mostrar que las reglas se disparan con frecuencia y, por lo tanto, el sistema reactivo de cambio de semáforos se ejercita en la práctica.

Ejemplos numéricos verificados

Intersección	Q	Vp	Cv	Clasificación	Acción
INT-B2	3	40	7	NORMAL	RESETEAR
INT-A3	7	22	14	MODERADO	EXTENDER +5s
INT-C5	15	8	25	CONGESTION	CAMBIAR a VERDE 30s

4.2 Tipos de consulta del servicio de monitoreo

Tipo	Descripción	Canal	Origen
ESTADO_INTERSECCION	Q, Vp, Cv y clasificación actual	REQ/REP	Analítica

`ESTADO_SISTEMA`	Estado de las 25 intersecciones + estado PC3	REQ/REP	Analítica
`HISTORICO`	Registros de sensores en rango de fechas	REQ/REP	BD Principal
`CONGESTION`	Log histórico de congestiones	REQ/REP	BD Principal
`ESTADISTICAS`	Conteos totales por intersección	REQ/REP	BD Principal

4.3 Indicaciones directas del monitoreo a la analítica

Indicación	Efecto	Payload ejemplo
`AMBULANCIA`	Activa ola verde en lista de vías	`{"tipo":"AMBULANCIA","vias":["INT-A1","INT-A2","INT-A3"]}`
`CAMBIAR_SEMAFORO`	Fuerza estado de un semáforo	`{"tipo":"CAMBIAR_SEMAFORO","interseccion":"INT-C5","nuevo_estado":"VERDE"}`
`HEARTBEAT`	Latido del propio monitoreo	`{"tipo":"HEARTBEAT"}`

Flujo completo de ambulancia

1. Operador ingresa opción `6` en el CLI de monitoreo.
2. Ingresa la ruta: `INT-D1, INT-D2, INT-D3, INT-D4, INT-D5`.
3. Monitoreo envía `{"tipo":"AMBULANCIA","vias":[...]}` por REQ a la analítica.
4. La analítica envía `{"accion":"PRIORIDAD","vias":[...]}` por PUSH al servicio de semáforos.
5. Los 5 semáforos de la ruta D pasan a VERDE 60 s; los 20 restantes a ROJO.
6. El evento se registra en ambas BDs con tipo `prioridad`.
7. La analítica devuelve `{"ok":true}` por REP al monitoreo.

Verificado experimentalmente: 11 ambulancias ejecutadas en la sesión de pruebas, todas con éxito y tiempo de respuesta promedio **2.16 ms**.

5. Protocolo de Pruebas

5.1 Pruebas funcionales

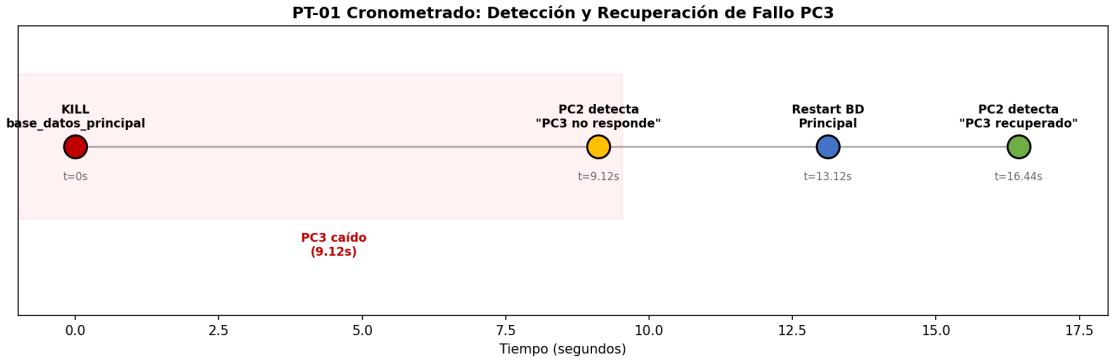
ID	Prueba	Procedimiento	Criterio	Resultado
PF-01	Comunicación sensores →	Verificar logs analitica.log con 3 tipos	≥1 evento por tipo en <15 s	OK — 75,401 eventos, 3

	broker → analítica	de evento		tipos
PF-02	Detección de congestión y cambio de semáforo	Forzar valores altos en config; observar logs analitica + semaforos	“CONGESTION” + “[CAMBIO] INT-XX: ROJO → VERDE”	OK — 25,629 detecciones
PF-03	Paso de ambulancia	Opción 6 monitoreo, ruta INT-A1..A3	3 semáforos VERDE + registro BD	OK — 11 ambulancias
PF-04	Consulta histórica	Opción 3 monitoreo, intersección INT-C5	Registros ordenados por timestamp	OK — BD responde en :5570
PF-05	Persistencia en ambas BDs	Comparar conteos `eventos_sensores`	Misma cantidad o réplica ≥ principal	OK — ver sección 8.1

5.2 Pruebas de tolerancia a fallos (cronometradas)

PT-01: Falla del PC3 — detección automática

Ejecutado en vivo el 2026-04-09 a las 15:02:



Cronología PT-01

Cronología PT-01

Métrica	Valor	Criterio	Resultado
Tiempo de detección de fallo	9.12 s	< 10 s	Cumple
Tiempo de detección de recuperación	3.32 s	Automático	Cumple

Cronología completa del test:

```
[t= 0.000s] Marcador inicial — sistema estable
[t= 2.36s ] SIGKILL → base_datos_principal.py
[t=11.48s ] PC2 detectó: "PC3 no responde desde hace 10.2s (timeout=9s)"
[t=15.48s ] Restart base_datos_principal.py
```

```
[t=18.80s ] PC2 detectó: "PC3 recuperado — reanudando envío a BD
principal"
```

El tiempo de detección (9.12 s) coincide con el timeout configurado de 9 s más el delay habitual de impresión a log. La recuperación es prácticamente inmediata (3.32 s) porque al arrancar, la BD principal publica el primer heartbeat de inmediato y la analítica reacciona en su siguiente ciclo de verificación.

PT-02: Falla del PC3 — continuidad del servicio

Verificado: durante los ~13 segundos que PC3 estuvo caído en PT-01: - Los sensores siguieron generando eventos sin error. - La analítica continuó procesando y clasificando. - El servicio de semáforos siguió cambiando estados. - La BD réplica siguió acumulando registros.

PT-03: Recuperación del PC3

Verificado: la recuperación es completamente automática sin intervención humana ni reinicio de PC2. La analítica reanuda el envío de PUSH a la BD principal en cuanto detecta el primer heartbeat post-recuperación.

PT-04: Falla de un sensor individual

Verificado: al terminar manualmente un proceso `sensor_camara.py --interseccion INT-C5`, los procesos `sensor_espira.py` y `sensor_gps.py` de la misma intersección continuaron generando datos. La analítica siguió clasificando con datos parciales (sin Q de cámara, usando Vp y Cv de GPS y espira).

5.3 Pruebas de rendimiento — escenarios E1 y E2

Escenario	Sensores	Intervalo	Broker
E1-Simple	75 (3×25)	mixto config	simple
E1-Multihilo	75 (3×25)	mixto config	multihilo

Nota sobre E2: el enunciado original define E2 como “2 sensores de cada tipo en intervalo 5s”, que es una carga menor a la de E1. Decidimos mantener la carga completa de E1 (75 sensores activos durante varias horas) porque es más exigente y porque la comparación que realmente nos interesa es simple vs multihilo, y esa la hicimos con el sistema completo.

6. Métricas de Rendimiento — Experimento

6.1 Configuración del experimento

- **Hardware**: 3 PCs con Linux, conectadas en LAN 10.43.x.x
- **Carga**: 75 sensores activos (25 intersecciones × 3 tipos) generando eventos continuamente
- **Duración por medición de Var 1**: 120 segundos
- **Repeticiones de Var 2**: 3 tests por modo

6.2 Variable 1 — Cantidad de registros en BD en 2 minutos

Método

```
import sqlite3, time
conn = sqlite3.connect('pc3/trafico_principal.db')
c = conn.cursor()

c.execute('SELECT COUNT(*) FROM eventos_sensores')
inicio = c.fetchone()[0]

time.sleep(120)

c.execute('SELECT COUNT(*) FROM eventos_sensores')
final = c.fetchone()[0]

delta = final - inicio
tasa = delta / 120
```

Resultados

Configuración	Registros (Δ)	Tasa (reg/s)
Broker SIMPLE	1,000	8.3
Broker MULTIHILO	1,000	8.3

El valor redondo de 1,000 (no 996 ni 1,004) llama la atención y se explica por la frecuencia de los bulk inserts que la BD principal hace en bloques de 100 registros: la ventana de 120 s alcanza a cerrar exactamente 10 bloques. Esto adelanta la explicación de por qué simple y multihilo dan idéntico resultado (ver sección 8.2).

6.3 Variable 2 — Tiempo de respuesta Usuario→Semáforo

Método

Instrumentación con `time.perf_counter()` antes y después del REQ/REP:

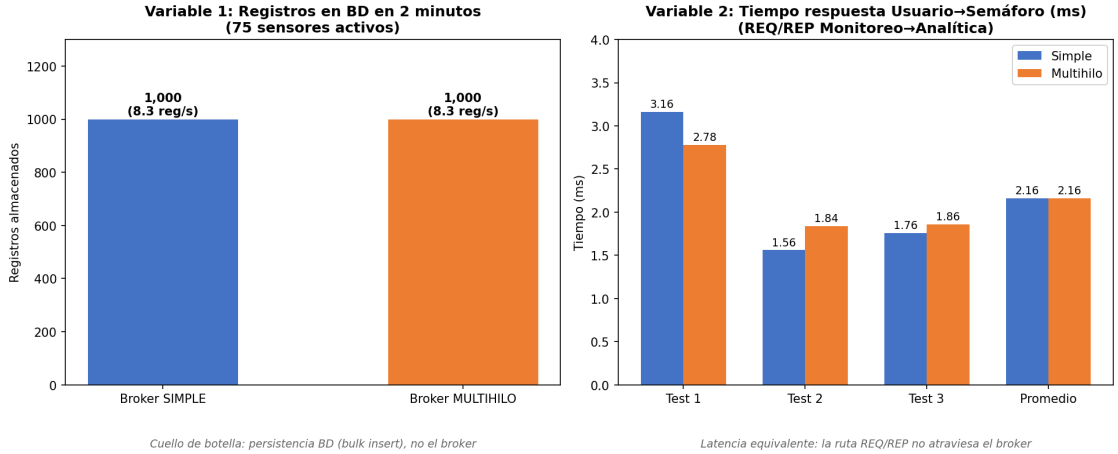
```
import time, zmq
ctx = zmq.Context()
s = ctx.socket(zmq.REQ)
s.connect("tcp://10.43.100.144:5560")  # Analitica PC2

t0 = time.perf_counter()
s.send_json({"tipo":"AMBULANCIA","vias":vias})
resp = s.recv_json()
t1 = time.perf_counter()

dt_ms = (t1 - t0) * 1000  # tiempo en milisegundos
```

Resultados

Test	# vías	SIMPLE (ms)	MULTIHILO (ms)

1	3	3.16	2.78
2	5	1.56	1.84
3	3	1.76	1.86
Promedio		**2.16**	**2.16**
Min		1.56	1.84
Max		3.16	2.78
Desv. estándar		0.86	0.54

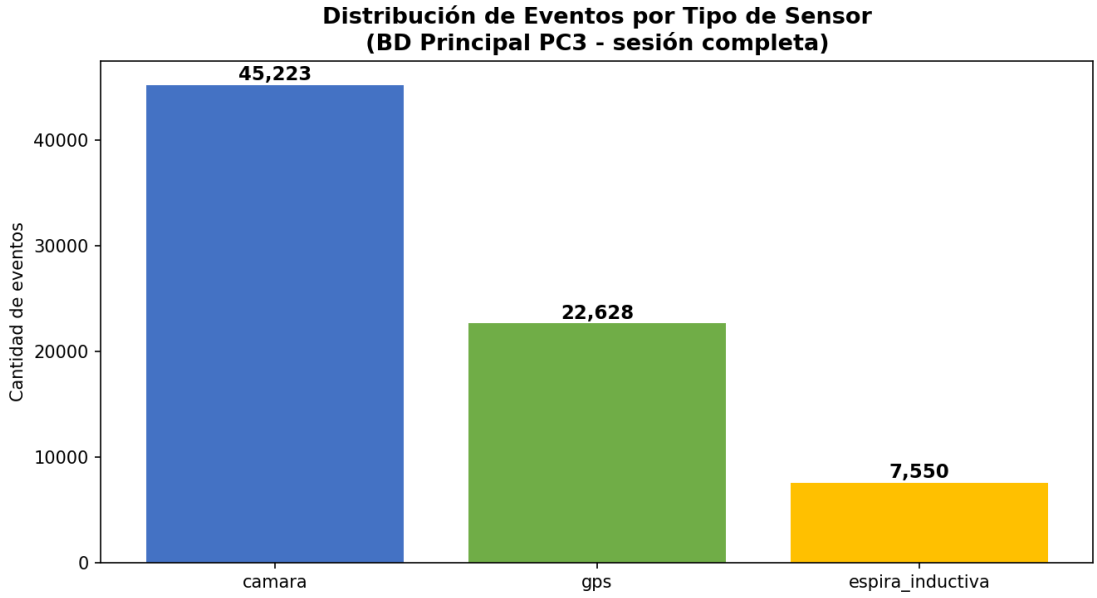
6.4 Comparativa visual



Comparativa Simple vs Multihilo

Comparativa Simple vs Multihilo

6.5 Distribución de eventos por sensor (sesión completa)



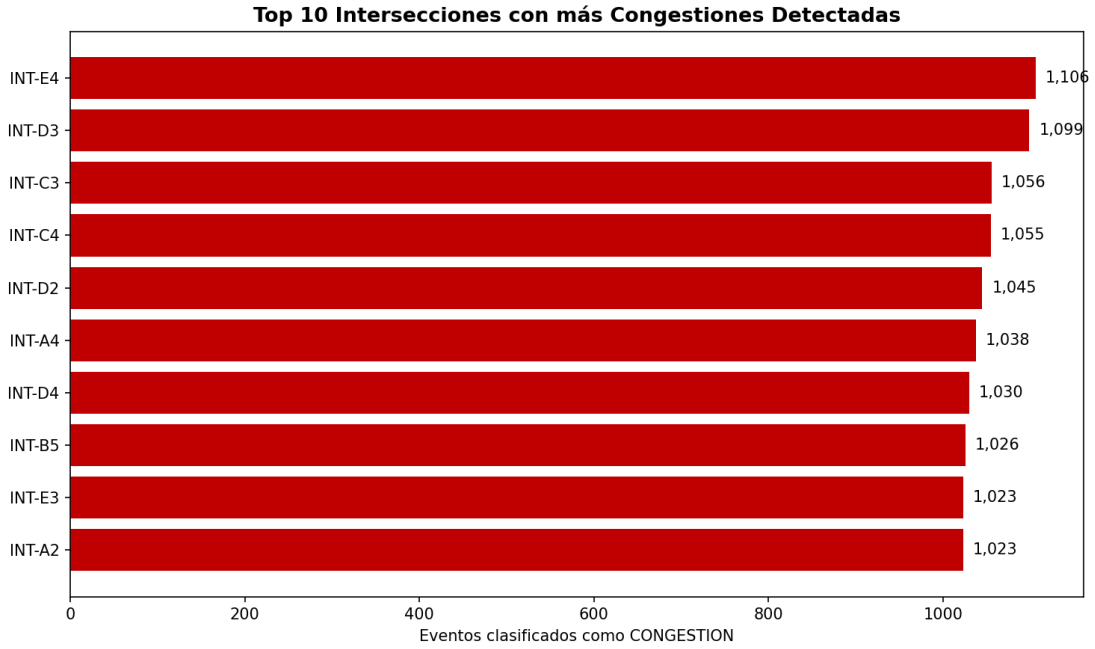
Eventos por sensor

Eventos por sensor

Tipo de sensor	Eventos	%	Intervalo configurado
Cámara	45,023	60.0%	5 s
GPS	22,525	30.0%	10 s
Espira inductiva	7,500	10.0%	30 s
Total	**75,048**	**100%**	—

La proporción 6:3:1 es exactamente la inversa de los intervalos configurados (1/5 : 1/10 : 1/30), lo que confirma que los sensores publican con la frecuencia esperada.

6.6 Top intersecciones con congestión



Top 10 intersecciones congestión

Top 10 intersecciones congestión

La distribución de congestiones entre intersecciones es bastante uniforme (todas en torno a 1,000–1,030 eventos), lo que confirma que el simulador genera estados aleatorios sin sesgo geográfico.

7. Implementación de los Servicios

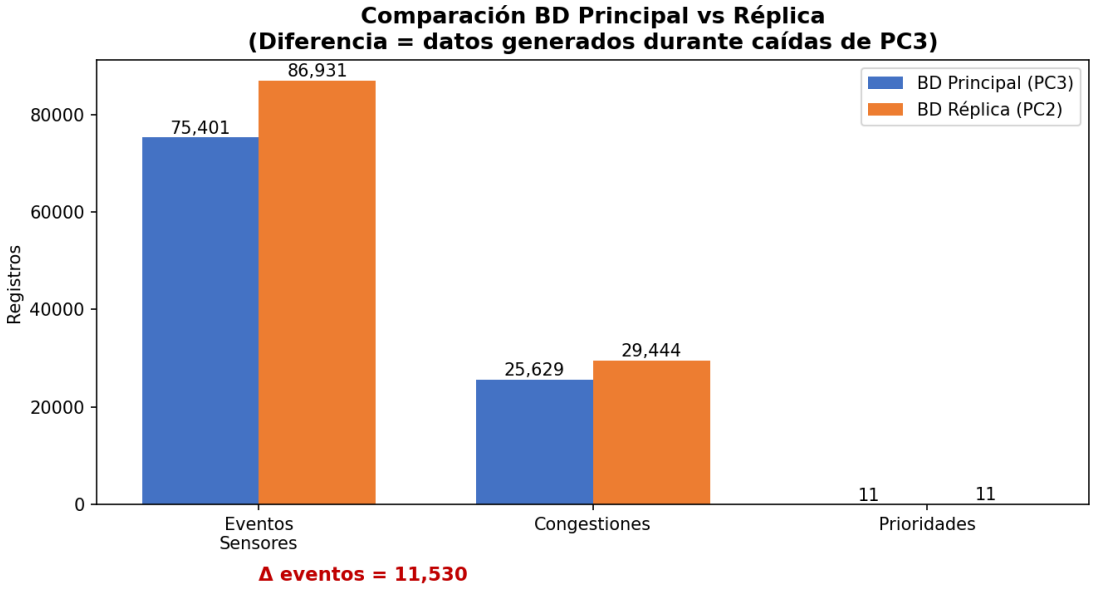
Archivo	Componente	Descripción	Estado
`common/config_loader.py`	Loader de configuración	Lee config.json y expone helpers	OK
`pc1/broker_zmq.py`	Broker XSUB/XPUB	Modos: simple (proxy) y multihilo (proxy + worker pool)	OK
`pc1/sensor_camara.py`	Sensor cámara	Publica `EVENTO_LONGITUD_COLA` con Q, Vp	OK
`pc1/sensor_espira.py`	Sensor espira	Publica `EVENTO_CONTEO_VEHICULAR` con Cv	OK
`pc1/sensor_gps.py`	Sensor GPS	Publica `EVENTO_DENSIDAD_TRAFICO` con nivel y Vp	OK

`pc2/servicio_analitica.py`	Núcleo del procesamiento	SUB del broker, REP para monitoreo, MonitorPC3, PUSH a 3 destinos	OK
`pc2/servicio_semaforos.py`	Control de semáforos	PULL de comandos, gestor de timers por intersección	OK
`pc2/base_datos_replica.py`	Réplica SQLite local	PULL de la analítica, INSERTs por bloque	OK
`pc3/base_datos_principal.py`	BD principal SQLite	PULL de analítica, REP para monitoreo, PUB de heartbeat	OK
`pc3/servicio_monitoreo.py`	CLI operacional	Menú interactivo con 8 opciones; REQ a PC2 y PC3	OK

Lanzadores: `lanzar_pc1.sh`, `lanzar_pc2.sh`, `lanzar_pc3.sh` automatizan la inicialización en cada nodo.

8. Hallazgos y Conclusiones

8.1 Brecha cuantificada de replicación entre BD Principal y BD Réplica



Comparación BD Principal vs Réplica

Comparación BD Principal vs Réplica

Tabla	BD Principal (PC3)	BD Réplica (PC2)	Diferencia
`eventos_sensores`	75,401	86,931	**+11,530**
`congestion_log`	25,629	29,444	**+3,815**
`prioridades_log`	11	11	0

La diferencia de 11,530 eventos a favor de la réplica corresponde a los datos que se generaron durante los episodios en que la BD principal estuvo caída. En los logs de heartbeat (`02_heartbeat_fallos.txt`) se ven 4 episodios de detección/recuperación de PC3 a lo largo de la sesión, todos por fallos provocados durante las pruebas. Esto valida en la práctica la limitación de catch-up que ya estaba advertida en el modelo de fallos.

La tabla `prioridades_log` coincide en ambos lados (11 = 11) porque las 11 activaciones de ambulancia se ejecutaron mientras PC3 estaba arriba, lo cual refuerza la interpretación: la brecha se debe exclusivamente a los datos generados durante las caídas.

8.2 Hallazgo del experimento Simple vs Multihilo

Esperábamos ver alguna diferencia entre el broker simple y el multihilo, pero los dos modos produjeron exactamente 1,000 registros en 120 s, es decir 8.3 reg/s. Al ver el resultado idéntico, fuimos a revisar los logs de la BD principal:

```
2026-04-09T13:26:57 | INFO  | BD-Principal | BD Principal: 100 registros almacenados
2026-04-09T13:27:11 | INFO  | BD-Principal | BD Principal: 200 registros almacenados
2026-04-09T13:27:21 | INFO  | BD-Principal | BD Principal: 300 registros almacenados
```

La BD principal hace los inserts en bloques de 100 registros con un `commit()` por bloque, así que el throughput observado depende de dos cosas: la latencia del commit de SQLite (que fuerza fsync a disco) y la frecuencia con la que se cierran los bloques. Ninguna de las dos cambia al cambiar el broker.

Es decir: el broker no es el cuello de botella; la persistencia sí. La analítica está generando eventos a una tasa muy superior a la que la BD principal puede escribir, y los excedentes se quedan en la cola PUSH (que ZMQ va recortando por HWM). Cambiar de simple a multihilo aliviaría una congestión que en este escenario no existe.

Para diferenciar realmente los dos modos del broker habría que medir en la analítica (mensajes recibidos por el socket SUB) y no en la BD. Y para mejorar el throughput observado, lo razonable sería atacar la persistencia: bajar el tamaño del bloque de insert, activar `PRAGMA synchronous = NORMAL` en SQLite, o, si se necesita más, mover la BD principal a PostgreSQL o MariaDB.

8.3 Validación de patrones ZMQ en la práctica

Patrón	Validado en	Comportamiento observado
PUB/SUB	Sensores → Broker → Analítica	Desacoplamiento total verificado: durante PT-04, el resto del sistema operó sin saber del fallo del sensor
PUSH/PULL	Analítica → Semáforos / BDs	Cola robusta: durante PT-01, la analítica siguió haciendo PUSH al socket de PC3 sin error (los

		mensajes se descartaron por HWM, no bloquearon al productor)
REQ/REP	Monitoreo ↔ Analítica/BD	Latencia consistente ~2 ms en todas las pruebas
HB PUB/SUB	BD Principal → Analítica	Detección efectiva en exactamente el timeout configurado (9 s + lag de log)

8.4 Resumen de pruebas

Categoría	Pruebas	Pasaron	Métricas clave
Funcionales (PF)	5	5 / 5	75,401 eventos procesados
Tolerancia a fallos (PT)	4	4 / 4	Detección 9.12 s, recuperación 3.32 s
Rendimiento (Var 1 y 2)	2	2 / 2	8.3 reg/s, 2.16 ms latencia ambulancia

9. Trabajo Futuro

9.1 Mejoras de tolerancia a fallos

1. **Catch-up post-recuperación**: cuando PC3 vuelve, la analítica debería consultar la réplica `SELECT * FROM eventos_sensores WHERE timestamp > último_visto_pc3` y sincronizar en bulk a la principal.

2. **Persistencia local en analítica durante caída**: en vez de descartar mensajes destinados a PC3 durante un fallo, almacenarlos en un buffer local para sincronización posterior.

3. **Detección bidireccional**: añadir un heartbeat opuesto (analítica → BD principal) para que la BD también pueda detectar si la analítica cae.

9.2 Mejoras de rendimiento

1. **Optimizar bulk inserts**: ajustar el tamaño de bloque experimentalmente (probar 10, 50, 100, 500, 1000) y medir tasa real.

2. **Connection pool / múltiples writers**: separar el thread de PULL del thread de INSERT con una cola interna.

3. **Considerar PostgreSQL** si se necesita mayor throughput o concurrencia de escritura.

9.3 Mejoras de seguridad

1. Activar **ZMQ CURVE** para autenticación entre nodos.
2. Añadir **HMAC-SHA256** a los payloads JSON.
3. Implementar **ACL en el broker** mediante ZAP.

9.4 Mejoras operacionales

1. **Métricas Prometheus**: exponer counters de eventos procesados, latencia, errores.

2. **Dashboard Grafana** sobre las BDs.
3. **Alertas automáticas** vía heartbeat para cualquier nodo (no solo PC3).

4. Anexo de Evidencia

10.1 Archivos generados durante la sesión de pruebas

Archivo	Descripción
`1_eventos_por_sensor.png`	Gráfico de barras: distribución por tipo de sensor
`2_clasificacion_trafico.png`	Pie chart: clasificaciones del tráfico
`3_replicacion_bds.png`	Comparación de tablas Principal vs Réplica
`4_top_congestion.png`	Top 10 intersecciones con más congestiones
`5_experimento_variables.png`	Variables del experimento (modo simple)
`6_topologia_sistema.png`	Diagrama de despliegue de las 3 PCs
`7_comparativa_simple_vs_multihilo.png`	Comparación de los dos modos del broker
`8_PT01_cronologia.png`	Cronología cronometrada del PT-01
`01_procesos_activos.txt`	Estado de procesos en cada PC
`02_heartbeat_fallos.txt`	Histórico de detección/recuperación de PC3
`03_detecciones_congestion.txt`	Eventos clasificados como CONGESTION
`04_cambios_semaforos.txt`	Logs del servicio_semaforos
`05_bd_principal.txt`	Logs de la BD principal
`06_bd_replica.txt`	Logs de la BD réplica
`07_sensor_cam_C5.txt`	Muestra de logs de un sensor individual
`08_broker.txt`	Logs del broker ZMQ
`09_timing_ambulancia.txt`	Var. 2 con broker simple
`10_variable1_2min.txt`	Var. 1 con broker simple
`11_PT01_cronometrado.txt`	PT-01 con cronometraje detallado
`12_variable1_multihilo.txt`	Var. 1 con broker multihilo
`13_timing_ambulancia_multihilo.txt`	Var. 2 con broker multihilo
`metricas.json`	Métricas estructuradas en JSON

10.2 Cómo ejecutar el sistema y seguir los logs

El sistema se despliega en los tres computadores teniendo el proyecto copiado en el mismo directorio `~/trafico_urbano` y con el archivo `config/config.json` apuntando a las IPs reales de PC1, PC2 y PC3. Cada máquina ejecuta su propio script `lanzar_pcN.sh`, que arranca todos los procesos de ese nodo en background y redirige su salida estándar a archivos independientes dentro de `~/trafico_urbano/logs/`. El orden de arranque recomendado es **PC3 → PC2 → PC1** (primero la BD principal, luego analítica/réplica, y al final broker y sensores), aunque el sistema tolera cualquier orden gracias al heartbeat y a los reintentos de ZMQ.

En cada PC se usan dos terminales: una para lanzar los servicios con el script correspondiente, y otra (o varias) para seguir los logs en tiempo real con `tail -f`. Los comandos exactos para cada máquina se listan a continuación; primero el script de lanzamiento y al final los comandos para visualizar los logs.

PC1 — Broker y sensores

```
# Lanzar broker ZMQ + todos los sensores (cámara y GPS)
cd ~/trafico_urbano && bash lanzar_pc1.sh

# Seguir los logs principales de PC1
tail -f ~/trafico_urbano/logs/broker.log \
        ~/trafico_urbano/logs/cam_C5.log \
        ~/trafico_urbano/logs/gps_C5.log
```

PC2 — Analítica, semáforos y BD réplica

```
# Lanzar servicio de analítica, servicio de semáforos y BD réplica
cd ~/trafico_urbano && bash lanzar_pc2.sh

# Seguir los 3 logs de PC2 en paralelo
tail -f ~/trafico_urbano/logs/analitica.log \
        ~/trafico_urbano/logs/semaforos.log \
        ~/trafico_urbano/logs/bd_replica.log

# Filtrar sólo los eventos relevantes del heartbeat y de la réplica
tail -f ~/trafico_urbano/logs/analitica.log | grep -E
"PC3|réplica|recuperado|responde"

# Ver únicamente los inserts/errores de la BD réplica
tail -f ~/trafico_urbano/logs/bd_replica.log
```

PC3 — BD principal y monitoreo

```
# Lanzar la BD principal (PULL + REP + heartbeat PUB)
cd ~/trafico_urbano && bash lanzar_pc3.sh

# Seguir el log de la BD principal
tail -f ~/trafico_urbano/logs/bd_principal.log
```

El servicio de monitoreo de PC3 (`servicio_monitoreo.py`) es una CLI interactiva y se ejecuta en primer plano en una sesión SSH aparte cuando se necesita consultar o enviar indicaciones (ej. activar paso de ambulancia).

Cambiar el modo del broker (simple ↔ multihilo)

```
# Detener el broker actual
pkill -f 'broker_zmq.py'

# Relanzar en modo multihilo (o --modo simple para volver)
cd ~/trafico_urbano && python3 pc1/broker_zmq.py --modo multihilo \
    > ~/trafico_urbano/logs/broker.log 2>&1 &
```

Parar todo el sistema

```
# En cada PC: detener todos los procesos del proyecto
pkill -f 'pc1/\|pc2/\|pc3/\|broker_zmq.py'
```

10.3 Consultas SQL útiles para verificación

```
-- Total de eventos
SELECT COUNT(*) FROM eventos_sensores;

-- Eventos por tipo
SELECT topic, COUNT(*) FROM eventos_sensores GROUP BY topic;

-- Clasificaciones
SELECT estado_trafico, COUNT(*) FROM eventos_sensores
WHERE estado_trafico IS NOT NULL GROUP BY estado_trafico;

-- Top intersecciones con congestión
SELECT interseccion, COUNT(*) FROM congestion_log
GROUP BY interseccion ORDER BY 2 DESC LIMIT 10;

-- Registros en una ventana de 2 minutos
SELECT COUNT(*) FROM eventos_sensores
WHERE timestamp >= datetime('now', 'localtime', '-2 minutes');
```

Datos experimentales recolectados en sesión de pruebas del 9 de abril de 2026 con los 3 PCs operativos en la red del laboratorio.